

Slate — design

DESIGN.md, exported 2026-07-21. Suckless PDF editor: view, annotate/markup, merge/split, redact, sign, form-fill, scan for sensitive content.

Why composition, not a new engine

Full ISO 32000 parity is a multi-year effort even for funded teams (PDF.js/MuPDF/Poppler-scale). The suckless answer isn't reinventing that — it's a thin app over libraries that already do each job well, matching this project's own "wrap the real tool, don't reimplement" doctrine (same pattern as Cairn's markdown skill).

Stack

- **GUI:** Tkinter (Python stdlib) — zero extra dependency for the toolkit itself.
- **Rendering / annotate / merge-split / forms:** PyMuPDF (`fitz`).
- **Redaction:** PyMuPDF's `apply_redactions()` , hardened (see below).
- **Redaction verification (independent second reader):** pikepdf (QPDF-backed — a different codebase than PyMuPDF/MuPDF, never let the same engine that wrote the file grade its own redaction).
- **Signing:** pyHanko (real PAdES B-B/B-T/B-LT/B-LTA, Acrobat-validation compatible — PyMuPDF's own signature-field support is basic/visual-only, not cryptographically real).
- **Encryption / password protection:** pikepdf.

Redaction — the hardened save path (non-negotiable)

PyMuPDF's destructive-removal guarantee is conditional: physical removal only holds "assuming `Document.save()` with a suitable garbage option." `io_pdf.py` 's save path after any redaction must always:

- `garbage=4, clean=True` — never expose a faster save that skips this; skipping it leaves orphaned, recoverable objects in the file.
- `incremental=False` (full rewrite) — a PDF with prior incremental-update revisions keeps pre-redaction content byte-recoverable in its own revision history regardless of how clean the current revision is.
- Strip XMP/document metadata as part of the same hardened save — same underlying risk (content surviving in the wrong place) as the incremental-history issue, not a separate feature.

Real, sourced bugs exist in `apply_redactions()` (PyMuPDF GitHub #3433, #2108, #3278 — over/under-redaction in certain content-stream layouts). The test harness must catch both directions, not just "is it

gone."

Signing — sequencing constraint

PDF signatures are append-only-incremental by design: signing must be the last write to a document. `io_pdf.py` must track signed-state and warn-or-refuse further edits to an already-signed document rather than silently invalidate the signature.

Forms — verified findings (slice 5, corrects the original plan)

Radio sibling-unset: the originally-cited gotcha is stale. The plan assumed PyMuPDF doesn't auto-unset sibling radio buttons and that `forms.py` would need to do it manually. Verified directly against the installed version (PyMuPDF 1.28.0): `Widget._checker()` already handles this correctly — setting one radio button's `field_value` to its own ON state and calling `.update()` automatically forces every sibling in the same field-name group back to `Off`. `forms.py` does nothing special; `tests/test_forms.py::test_radio_sibling_auto_unset` pins this behavior so a future PyMuPDF upgrade that regresses it gets caught immediately.

Real gotcha instead: radio GROUP creation, and widget page-lifetime. PyMuPDF cannot currently *create* a new interlinked radio group — adding two `Widget()` objects with a shared `field_name` via `add_widget()` raises `ValueError: bad xref` (confirmed, matches PyMuPDF GitHub discussion #2333: group creation isn't supported, only filling an existing group's values is). Not a blocker for v1 — Slate fills received forms, it doesn't author new ones from scratch — but the test fixture's radio group had to be built at the raw PDF-object level via `pikepdf` instead. Separately: a `Widget` holds only a *weak* reference to its parent page; letting that page object go out of scope while still holding widgets turns a later sibling-unset into a raw `ReferenceError`, not a clean update. Always keep the page bound to a named variable for as long as any of its widgets are being read or written.

Build order (each slice has its own standalone pass/fail check)

Redaction is proven right after the minimum viewer needed to see anything — early, not deferred to "once the UI feels done."

#	Slice	Check
0	Skeleton: venv, fossil repo, Tkinter window opens a fixture PDF	Window opens, no exceptions, page count matches
1	Minimal viewer: render, nav, zoom	Every page of a fixture renders; checksums match expected

2	Redaction core + verification harness (the trust gate)	Plant a canary string + image in a fixture; redact; assert gone via PyMuPDF's <code>get_text()</code> AND a pikepdf raw-object scan (incl. orphaned objects) AND image byte-hash absent — on both a plain fixture and one with pre-existing incremental-update history — and explicitly assert the <i>unsafe</i> save path still shows the canary, proving the safe path is load-bearing
3	Merge/split/reorder/extract	Round-trip page count + per-page text-hash match
4	Annotate (highlight, freetext, ink, shapes, stamp)	Add one of each, reload, enumerate <code>page.annots()</code> , confirm type/rect/contents
5	Forms fill	One of each widget type; set, reload, confirm persisted; cross-check with an independent renderer; verify radio-sibling unsetting
6	Signing (pyHanko, PAdES B-B)	Sign with a self-signed test cert; verify via pyHanko's own independent validation call against the saved file
7	Security/encryption	Round-trip: correct password opens, wrong password fails, permission bits actually enforced
8	UI integration	One real end-to-end task per feature category, single sitting, no restarts

Scope decisions

OUT of v1: XFA (Adobe LiveCycle) forms (detect and warn, don't silently mis-save). PDF/A validation/conversion (unless a real compliance need surfaces — wrap veraPDF later, don't hand-roll). OCR for scanned PDFs (image-region redaction still works without it).

IN v1: opening and producing password-protected PDFs. A mandatory safe-save policy — never overwrite the original in place; auto-backup or save-as-copy before any destructive op, especially redaction.

Business question, not engineering: PAdES B-LT/B-LTA needs a real non-self-signed cert to matter to external recipients — v1 ships B-B (fine for internal sign-off), B-LT/LTA only worth building once a real signing cert exists.

Scan for sensitive content (`scan.py`)

Started as a one-off scratch audit script (Devin asked to scan his real Downloads folder for anything needing redaction), promoted to a real feature after it found genuine sensitive content in a real file (a bank account + routing number in an actual UMB account-verification letter) and, separately, had a real false-negative during its own development.

Scope: number-shaped financial/PII patterns only — SSN (`123-45-6789`), ABA routing numbers, labeled account numbers, and Luhn-valid credit card numbers. Does NOT catch general PII (a resume's

phone/address with no account-shaped context), business-confidential content, or anything on an image-only/scanned page — same OCR gap already named out-of-v1 for redaction itself. A page with 0 extractable characters is reported as its own `unscannable` finding rather than silently folded into "nothing found" — those are different claims, and conflating them is exactly the bug this project caught in itself (next paragraph).

Real bug, caught auditing Devin's actual Downloads folder, not a hypothetical: the first version matched a label and its value on the *same line* (`Account Number:\s*(\d+)`). Real PDFs routinely don't lay text out that way — the bank letter's own extracted text put `Account Number:` , a blank line, and `9825039777` as three separate lines. The same-line version silently reported the real file clean. Fixed by scanning forward a few non-blank lines after a label match instead of requiring the value on the same line; pinned as its own named regression test (`test_label_and_value_on_separate_lines_regression`) building that exact three-line layout, not folded into a general-case test.

UI wiring: Edit → "Scan this document..." runs `scan.scan_document` against the open file and offers to mark every hit with a resolvable rect as a pending redaction in one click — scan and redact are meant to chain together, not be two disconnected features. File → "Scan folder for sensitive PDFs..." reuses `scan.scan_directory` for the batch-audit case (the actual real-world use this started from).

Real bug found wiring this into the UI, unrelated to scan.py itself: `_on_release` 's redact-mode branch called a blocking `messagebox.showinfo` on *every single drag* — meant as "region marked" feedback, but it made a multi-region redaction pass annoying (a modal popup per mark) and, worse, made this exact code path hang in an automated test once nothing was there to dismiss the dialog. Fixed by removing the popup entirely — the status bar (`render()`) already shows the pending-redaction count for the current page, which is the correct non-blocking feedback.

Fixtures

`tests/fixtures/` holds small synthetic PDFs only — generated programmatically, never real documents. One fixture must include pre-existing incremental-update history (to catch the redaction/history gap above) and one must include a pre-existing tag tree (to confirm redact/merge/annotate don't silently strip it, even though authoring tagged PDF is out of v1).